WEEK – 6

TASK 1 :To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

ramya@LAPTOP-OPDAF309:

GNU nano 8.7.1 task1.c *

```c
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_CMD 100

int main() {
    char history[MAX_HISTORY][MAX_CMD];
    int count = 0;
    char command[MAX_CMD];

    printf("Enter commands (type 'exit' to stop):\n");

    while (1) {
        printf("Command: ");
        fgets(command, MAX_CMD, stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (count < MAX_HISTORY) {
            strcpy(history[count], command);
            count++;
        } else {
            for (int i = 1; i < MAX_HISTORY; i++)
                strcpy(history[i - 1], history[i]);

            strcpy(history[MAX_HISTORY - 1], command);
        }
    }

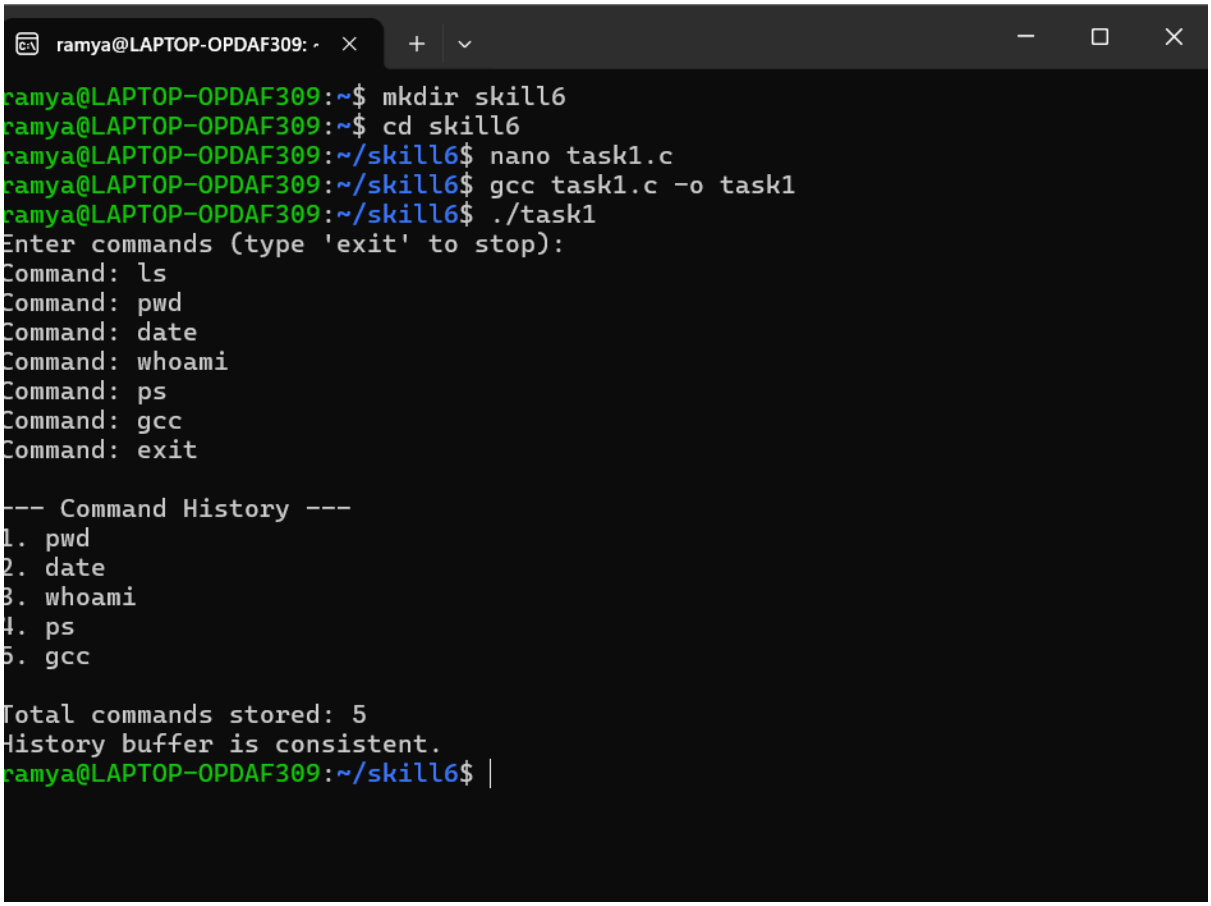
    printf("\n--- Command History ---\n");

    for (int i = 0; i < count; i++)
        printf("%d. %s\n", i + 1, history[i]);

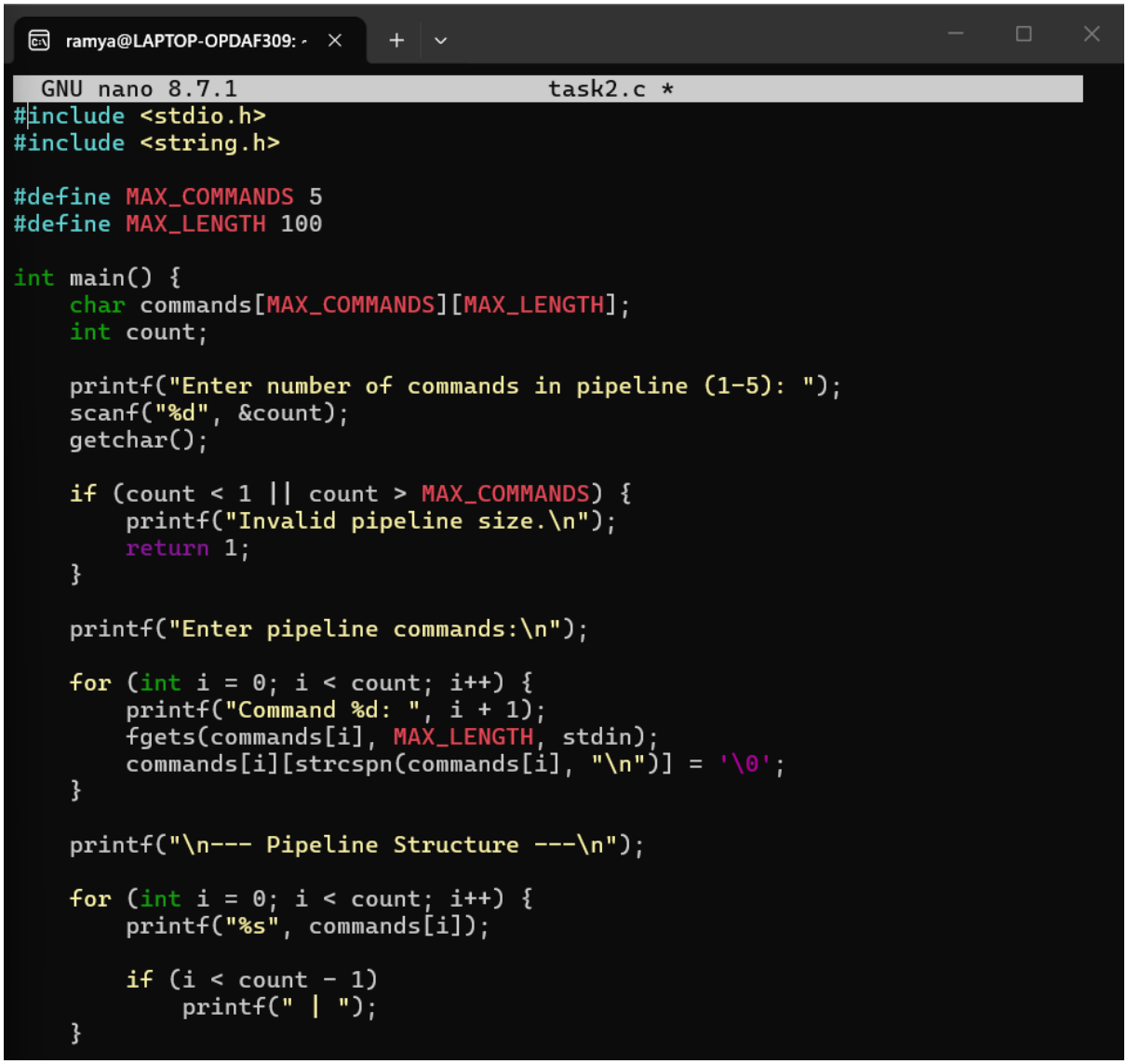
    printf("\nTotal commands stored: %d\n", count);

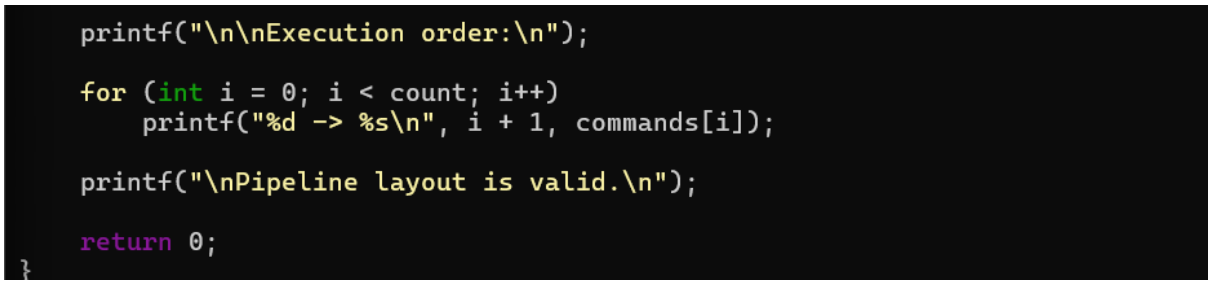
    if (count <= MAX_HISTORY)
        printf("History buffer is consistent.\n");
    else
        printf("History buffer overflow.\n");

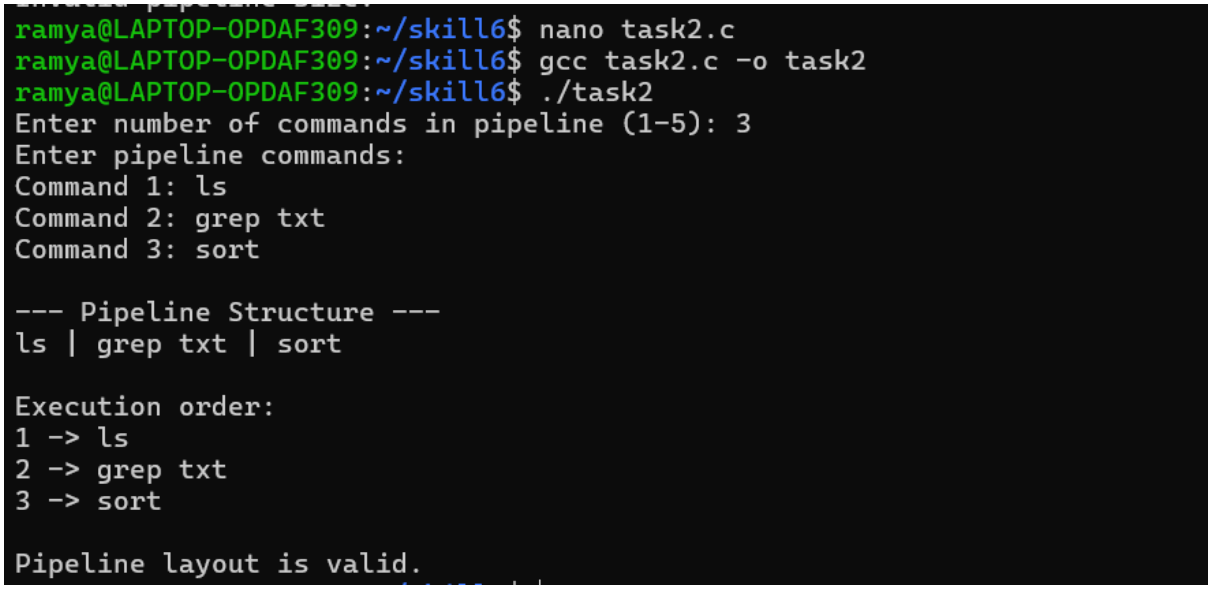
    return 0;
}
```



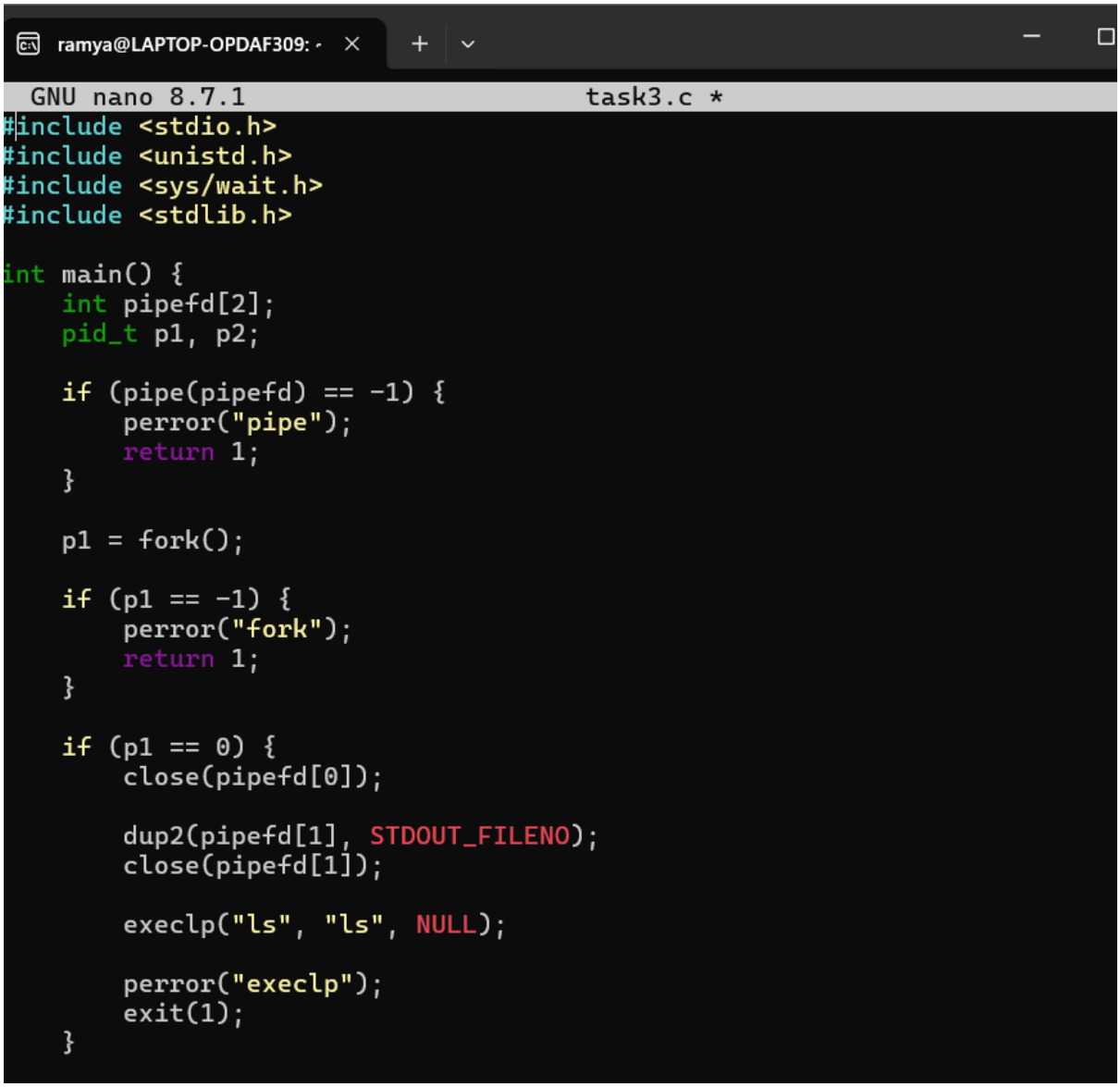
TASK 2 : To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations


ramya@LAPTOP-OPDAF309:
GNU nano 8.7.1 task2.c *
#include <stdio.h>
#include <string.h>
#define MAX_COMMANDS 5
#define MAX_LENGTH 100
int main() {
char commands[MAX_COMMANDS][MAX_LENGTH];
int count;
printf("Enter number of commands in pipeline (1-5): ");
scanf("%d", &count);
getchar();
if (count < 1 || count > MAX_COMMANDS) {
printf("Invalid pipeline size.\n");
return 1;
}
printf("Enter pipeline commands:\n");
for (int i = 0; i < count; i++) {
printf("Command %d: ", i + 1);
fgets(commands[i], MAX_LENGTH, stdin);
commands[i][strcspn(commands[i], "\n")] = '\0';
}
printf("\n--- Pipeline Structure ---\n");
for (int i = 0; i < count; i++) {
printf("%s", commands[i]);
if (i < count - 1)
printf(" | ");
}


printf("\n\nExecution order:\n");
for (int i = 0; i < count; i++)
printf("%d -> %s\n", i + 1, commands[i]);
printf("\nPipeline layout is valid.\n");
return 0;
}



TASK 3 : To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow


ramya@LAPTOP-OPDAF309:
GNU nano 8.7.1 task3.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main() {
int pipefd[2];
pid_t p1, p2;
if (pipe(pipefd) == -1) {
perror("pipe");
return 1;
}
p1 = fork();
if (p1 == -1) {
perror("fork");
return 1;
}
if (p1 == 0) {
close(pipefd[0]);
dup2(pipefd[1], STDOUT_FILENO);
close(pipefd[1]);
execlp("ls", "ls", NULL);
perror("execlp");
exit(1);
}

```c
    p2 = fork();

    if (p2 == -1) {
        perror("fork");
        return 1;
    }

    if (p2 == 0) {
        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);

        execlp("wc", "wc", "-l", NULL);

        perror("execlp");
        exit(1);
    }

    close(pipefd[0]);
    close(pipefd[1]);

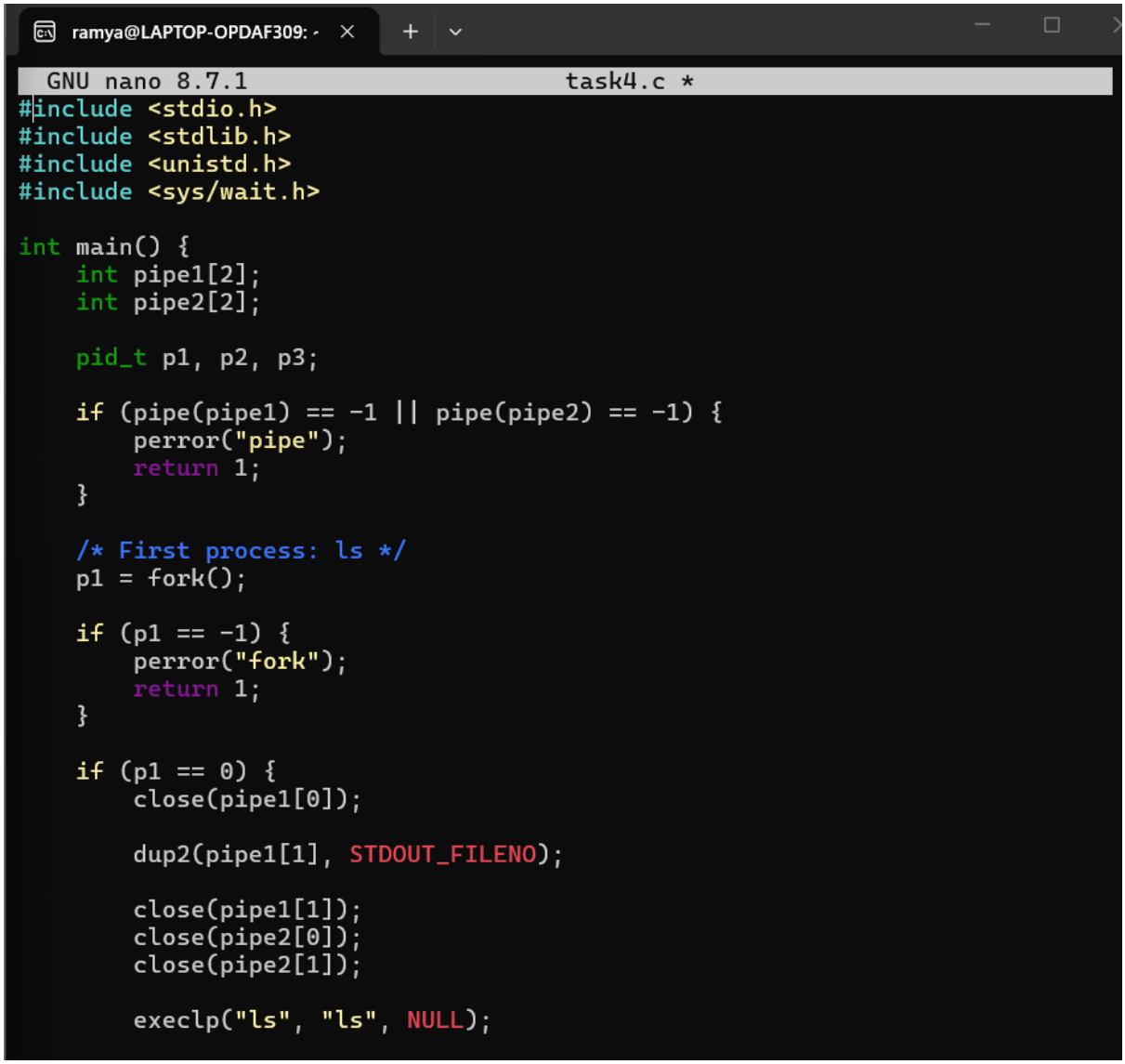
    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);

    printf("Pipeline execution completed.\n");

    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/skill6$ nano task3.c
ramya@LAPTOP-OPDAF309:~/skill6$ gcc task3.c -o task3
ramya@LAPTOP-OPDAF309:~/skill6$ ./task3
6
Pipeline execution completed.
ramya@LAPTOP-OPDAF309:~/skill6$
```

TASK 4 : To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines


ramya@LAPTOP-OPDAF309:
GNU nano 8.7.1 task4.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
int pipe1[2];
int pipe2[2];
pid_t p1, p2, p3;
if (pipe(pipe1) == -1 || pipe(pipe2) == -1) {
perror("pipe");
return 1;
}
/* First process: ls */
p1 = fork();
if (p1 == -1) {
perror("fork");
return 1;
}
if (p1 == 0) {
close(pipe1[0]);
dup2(pipe1[1], STDOUT_FILENO);
close(pipe1[1]);
close(pipe2[0]);
close(pipe2[1]);
execlp("ls", "ls", NULL);

```
        perror("ls");
        exit(1);
    }

    /* Second process: grep */
    p2 = fork();

    if (p2 == -1) {
        perror("fork");
        return 1;
    }

    if (p2 == 0) {
        close(pipe1[1]);
        close(pipe2[0]);

        dup2(pipe1[0], STDIN_FILENO);
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe2[1]);

        execlp("grep", "grep", ".c", NULL);

        perror("grep");
        exit(1);
    }

    /* Third process: wc */
    p3 = fork();

    if (p3 == -1) {
        perror("fork");
        return 1;
    }
```

```c
    if (p3 == 0) {
        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[1]);

        dup2(pipe2[0], STDIN_FILENO);

        close(pipe2[0]);

        execlp("wc", "wc", "-l", NULL);

        perror("wc");
        exit(1);
    }

    /* Parent closes all pipe descriptors */
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    /* Wait for all processes */
    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);
    waitpid(p3, NULL, 0);

    printf("Multiple pipeline execution completed.\n");

    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/skill6$ nano task4.c
ramya@LAPTOP-OPDAF309:~/skill6$ gcc task4.c -o task4
ramya@LAPTOP-OPDAF309:~/skill6$ ./task4
4
Multiple pipeline execution completed.
ramya@LAPTOP-OPDAF309:~/skill6$
```